

College Football Prediction Model

Christian Etherson

2026-08-05

Contents

1	Project Description	1
2	Loading Data and Feature Engineering	2
3	Test and Train Split	8
4	Logistic Model Training	8
4.1	Binned Probabilities and Confusion Matrix for Full Non MOV Truncated Model	13
4.2	Binned Probabilities and Confusion Matrix for Full MOV Truncated Model	16
4.3	Binned Probabilities and Confusion Matrix for Reduced Non MOV Truncated Model	19
5	Decision Tree Model	23
6	K Nearest Neighbors	25
7	Model Accuracy Comparison	28
8	Championship Game Predictions with the Reduced Truncated MOV Model	29

1 Project Description

My final project will be a continuation of the college football rankings project. In that project, I created a ranking system using logistic regression and ranking based on overall

win percentages. In this project, I seek to expand on the prediction aspect and try to devise a model with the highest prediction accuracy. To do this, I will try to optimize logistic regression as well as try other models such as K Nearest Neighbors or Decision Trees. The goal of the project will be to find the model with the highest prediction accuracy and predict the 6 championship games happening this weekend.

2 Loading Data and Feature Engineering

This project pulls data from raw college football box scores from 2010 to 2024. To predict game outcomes without leaking information from the game being predicted, I engineered features that summarize each team’s performance prior to the matchup rather than using in-game statistics directly. For each team-season, I calculated per-game efficiency metrics (passing and rushing yards per attempt, third-down conversion rate, penalty yards per penalty), volume metrics (first downs per game), and outcome metrics (turnovers committed and forced per game, average margin of victory, yards allowed) by reshaping the box scores into a team-game format and aggregating by season. These season-level averages were then joined back to each matchup and differenced (home minus away) to create relative strength indicators—for example, `pass_eff_diff_season` captures whether the home team’s season-long passing efficiency exceeds the away team’s. Because each team’s season average is computed across all of its games in that season, including the one being predicted, this approach dilutes rather than fully eliminates data leakage—each individual game contributes only a small fraction to its own season average (roughly $1/12$ for a typical season), but the leakage is not entirely removed. I also created a truncated variant of margin of victory, capping per-game scoring differentials at ± 4 points before averaging into the season-level statistic, to address the fact that margin of victory can be disproportionately inflated by a small number of blowout wins or losses—a team that wins many close games should be distinguishable from one that wins occasionally by 50 points. This differential, season-based approach was designed so that features would reflect teams’ demonstrated tendencies rather than the specific game outcome being predicted, substantially reducing (though not fully eliminating) data leakage while still capturing the relative matchup dynamics that drive win probability.

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

```
cfbData <- read.csv("cfb_box-scores_2002-2024.csv")
```

```
#Filters for games after 2009 and removes irrelevant features, and  
#creates game-level features.
```

```
cfbData <- cfbData %>%
```

```
  filter(season >= 2010) %>%
```

```
  select(-date, -time_et, -game_type, -rank_away, -rank_home, -conf_away, -conf_home,  
         -neutral, -q1_away, -q2_away, -q3_away, -q4_away, -ot_away, -q1_home, -q2_home,  
         -q3_home, -q4_home, -ot_home, -attendance, -tv) %>%
```

```
  mutate(  
    home_win = ifelse(score_home > score_away, 1, 0),  
    turnovers_caused_home = fum_away + int_away,  
    turnovers_caused_away = fum_home + int_home,  
    fd_diff = first_downs_home - first_downs_away,  
    third_down_comp_diff = third_down_comp_home - third_down_comp_away,  
    third_down_att_diff = third_down_att_home - third_down_att_away,  
    fourth_down_comp_diff = fourth_down_comp_home - fourth_down_comp_away,  
    fourth_down_att_diff = fourth_down_att_home - fourth_down_att_away,
```

```

pass_comp_diff = pass_comp_home - pass_comp_away,
pass_att_diff = pass_att_home - pass_att_away,
pass_yards_diff = pass_yards_home - pass_yards_away,
rush_att_diff = rush_att_home - rush_att_away,
rush_yards_diff = rush_yards_home - rush_yards_away,
total_yards_diff = total_yards_home - total_yards_away,
fum_diff = fum_home - fum_away,
int_diff = int_home - int_away,
pen_num_diff = pen_num_home - pen_num_away,
pen_yards_diff = pen_yards_home - pen_yards_away,
possession_diff = possession_home - possession_away,
home_yards_allowed = total_yards_away,
away_yards_allowed = total_yards_home
)

# Creates game-level efficiency data for passing, rushing, and penalties

cfbData <- cfbData %>%
  mutate(
    pass_eff_home = ifelse(pass_att_home == 0, 0, pass_yards_home / pass_att_home),
    pass_eff_away = ifelse(pass_att_away == 0, 0, pass_yards_away / pass_att_away),
    rush_eff_home = ifelse(rush_att_home == 0, 0, rush_yards_home / rush_att_home),
    rush_eff_away = ifelse(rush_att_away == 0, 0, rush_yards_away / rush_att_away),
    pen_eff_home = ifelse(pen_num_home == 0, 0, pen_yards_home / pen_num_home),
    pen_eff_away = ifelse(pen_num_away == 0, 0, pen_yards_away / pen_num_away),
    third_down_rate_home = ifelse(third_down_att_home == 0, 0,
                                   third_down_comp_home / third_down_att_home),
    third_down_rate_away = ifelse(third_down_att_away == 0, 0,
                                   third_down_comp_away / third_down_att_away),
    mov_home = score_home - score_away,
    mov_away = score_away - score_home
  )

# Separates the game-level data into home game and away game stats

```

```

home_stats <- cfbData %>%
  select(season, team = home, pass_eff_home, rush_eff_home, pen_eff_home, first_downs_home,
         third_down_rate_home, fum_home, int_home, mov_home, home_yards_allowed,
         turnovers_caused_home) %>%
  rename(pass_eff = pass_eff_home, rush_eff = rush_eff_home, pen_eff = pen_eff_home,
         first_downs = first_downs_home, third_down_rate = third_down_rate_home,
         fum = fum_home, int = int_home, mov = mov_home, yards_allowed = home_yards_allowed,
         turnovers_caused = turnovers_caused_home)

away_stats <- cfbData %>%
  select(season, team = away, pass_eff_away, rush_eff_away, pen_eff_away, first_downs_away,
         third_down_rate_away, fum_away, int_away, mov_away, away_yards_allowed,
         turnovers_caused_away) %>%
  rename(pass_eff = pass_eff_away, rush_eff = rush_eff_away, pen_eff = pen_eff_away,
         first_downs = first_downs_away, third_down_rate = third_down_rate_away,
         fum = fum_away, int = int_away, mov = mov_away, yards_allowed = away_yards_allowed,
         turnovers_caused = turnovers_caused_away)

#Combines the home and away game statistics into overall team statistics

team_stats <- bind_rows(home_stats, away_stats)

#Groups the team statistics by season to create season-wide efficiency statistics

season_eff_stats <- team_stats %>%
  group_by(season, team) %>%
  summarize(pass_eff_season = mean(pass_eff),
            rush_eff_season = mean(rush_eff),
            pen_eff_season = mean(pen_eff),
            first_downs_per_game = mean(first_downs),
            third_down_rate_season = mean(third_down_rate),
            turnovers_pg_season = mean(fum + int),
            mov_season = mean(mov),

```

```

yards_allowed_season = mean(yards_allowed),
turnovers_caused_pg_season = mean(turnovers_caused),
.groups = "drop")

#Combines the season-level data into the dataset and creates difference
#statistics for each game

cfbData <- cfbData %>%
  left_join(season_eff_stats, by = c("season", "home" = "team")) %>%
  rename(pass_eff_season_home = pass_eff_season,
         rush_eff_season_home = rush_eff_season,
         pen_eff_season_home = pen_eff_season,
         first_downs_per_game_home = first_downs_per_game,
         third_down_rate_season_home = third_down_rate_season,
         turnovers_pg_season_home = turnovers_pg_season,
         mov_season_home = mov_season,
         yards_allowed_season_home = yards_allowed_season,
         turnovers_caused_pg_season_home = turnovers_caused_pg_season) %>%
  left_join(season_eff_stats, by = c("season", "away" = "team")) %>%
  rename(pass_eff_season_away = pass_eff_season,
         rush_eff_season_away = rush_eff_season,
         pen_eff_season_away = pen_eff_season,
         first_downs_per_game_away = first_downs_per_game,
         third_down_rate_season_away = third_down_rate_season,
         turnovers_pg_season_away = turnovers_pg_season,
         mov_season_away = mov_season,
         yards_allowed_season_away = yards_allowed_season,
         turnovers_caused_pg_season_away = turnovers_caused_pg_season) %>%
  mutate(pass_eff_diff_season = pass_eff_season_home - pass_eff_season_away,
         rush_eff_diff_season = rush_eff_season_home - rush_eff_season_away,
         pen_eff_diff_season = pen_eff_season_home - pen_eff_season_away,
         first_downs_diff_season =
           first_downs_per_game_home - first_downs_per_game_away,
         third_down_rate_diff_season =

```

```

    third_down_rate_season_home - third_down_rate_season_away,
turnovers_pg_diff_season =
    turnovers_pg_season_home - turnovers_pg_season_away,
mov_diff_season = mov_season_home - mov_season_away,
yards_allowed_diff_season =
    yards_allowed_season_home - yards_allowed_season_away,
turnovers_caused_pg_diff =
    turnovers_caused_pg_season_home - turnovers_caused_pg_season_away)

cfbData <- cfbData %>%
  select(season, home, away, home_win, pass_eff_home, rush_eff_home, home_yards_allowed,
    rush_eff_diff_season, first_downs_diff_season, third_down_rate_diff_season,
    turnovers_pg_diff_season, mov_diff_season, yards_allowed_diff_season,
    turnovers_caused_pg_diff)

#Creates the truncated margin of victory difference statistic

cfbData <- cfbData %>%
  mutate(
    mov_trunc_home = pmin(pmax(mov_home, -4), 4),
    mov_trunc_away = pmin(pmax(mov_away, -4), 4),
    mov_diff_trunc = mov_trunc_home - mov_trunc_away
  )

team_season_mov <- bind_rows(
  cfbData %>% select(season, team = home, mov_trunc = mov_trunc_home),
  cfbData %>% select(season, team = away, mov_trunc = mov_trunc_away)
) %>%
  group_by(season, team) %>%
  summarise(mov_trunc_season = mean(mov_trunc), .groups = "drop")

cfbData <- cfbData %>%
  left_join(team_season_mov, by = c("season", "home" = "team")) %>%
  rename(mov_trunc_season_home = mov_trunc_season) %>%

```

```

left_join(team_season_mov, by = c("season", "away" = "team")) %>%
rename(mov_trunc_season_away = mov_trunc_season) %>%
mutate(mov_trunc_diff_season = mov_trunc_season_home - mov_trunc_season_away)

cfbData <- cfbData %>% na.omit()

```

3 Test and Train Split

I separated the dataset into a train split of data from 2010 to 2022 and a test set of the 2023 and 2024 seasons. This creates a roughly 15/85 test train split.

```

test_season <- 2023

train_data <- subset(cfbData, season < test_season)
test_data <- subset(cfbData, season %in% c(2023, 2024))

```

4 Logistic Model Training

The first statistical method I will use is logistic regression. I will try three different models of this type. The first includes all differenced season-level features and does not truncate differenced margin of victory, the second also includes all differenced season-level features but truncates the margin of victory difference between (-4,4), and the final model reduces the number of features of the first two and also includes the truncated margin of victory difference.

```

#Full non MOV truncated logistic regression model

season_glm_full <- glm(
  home_win ~ pass_eff_diff_season +
    rush_eff_diff_season +
    turnovers_pg_diff_season +
    pen_eff_diff_season +
    first_downs_diff_season +

```



```

        third_down_rate_diff_season +
        mov_diff_season +
        yards_allowed_diff_season +
        turnovers_caused_pg_diff,
    data = train_data,
    family = binomial
)

summary(season_glm_full)

```

```

##
## Call:
## glm(formula = home_win ~ pass_eff_diff_season + rush_eff_diff_season +
##      turnovers_pg_diff_season + pen_eff_diff_season + first_downs_diff_season +
##      third_down_rate_diff_season + mov_diff_season + yards_allowed_diff_season +
##      turnovers_caused_pg_diff, family = binomial, data = train_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      0.4456505   0.0263584   16.907  < 2e-16 ***
## pass_eff_diff_season    0.0124125   0.0214470    0.579  0.56276
## rush_eff_diff_season   -0.0381855   0.0283397   -1.347  0.17784
## turnovers_pg_diff_season -0.1363687   0.0477409   -2.856  0.00428 **
## pen_eff_diff_season     0.0375827   0.0247149    1.521  0.12835
## first_downs_diff_season -0.0166286   0.0108341   -1.535  0.12482
## third_down_rate_diff_season 1.2702614   0.4766477    2.665  0.00770 **
## mov_diff_season         0.1221203   0.0051338   23.787  < 2e-16 ***
## yards_allowed_diff_season 0.0008160   0.0005814    1.403  0.16048
## turnovers_caused_pg_diff 0.0648543   0.0510353    1.271  0.20381
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##

```

```
## Null deviance: 14236.2 on 10698 degrees of freedom
## Residual deviance: 9047.4 on 10689 degrees of freedom
## AIC: 9067.4
##
## Number of Fisher Scoring iterations: 6
```

#Full MOV truncated logistic regression model

```
season_glm_MOVTrunc <- glm(
  home_win ~ pass_eff_diff_season +
    rush_eff_diff_season +
    turnovers_pg_diff_season +
    pen_eff_diff_season +
    first_downs_diff_season +
    third_down_rate_diff_season +
    mov_trunc_diff_season +
    yards_allowed_diff_season +
    turnovers_caused_pg_diff,
  data = train_data,
  family = binomial
)
```

```
summary(season_glm_MOVTrunc)
```

```
##
```

```
## Call:
```

```
## glm(formula = home_win ~ pass_eff_diff_season + rush_eff_diff_season +
##     turnovers_pg_diff_season + pen_eff_diff_season + first_downs_diff_season +
##     third_down_rate_diff_season + mov_trunc_diff_season + yards_allowed_diff_season +
##     turnovers_caused_pg_diff, family = binomial, data = train_data)
```

```
##
```

```
## Coefficients:
```

	Estimate	Std. Error	z value	Pr(> z)
## (Intercept)	0.4672720	0.0281720	16.586	<2e-16 ***
## pass_eff_diff_season	-0.0053139	0.0223759	-0.237	0.8123

```
## rush_eff_diff_season      -0.0063592  0.0290685  -0.219   0.8268
## turnovers_pg_diff_season  -0.0112243  0.0514347  -0.218   0.8273
## pen_eff_diff_season       -0.0037710  0.0272400  -0.138   0.8899
## first_downs_diff_season    0.0052167  0.0105853   0.493   0.6221
## third_down_rate_diff_season -0.2105313  0.5281348  -0.399   0.6902
## mov_trunc_diff_season      0.9467050  0.0277814  34.077  <2e-16 ***
## yards_allowed_diff_season  0.0002932  0.0004937   0.594   0.5526
## turnovers_caused_pg_diff   -0.0876201  0.0530425  -1.652   0.0986 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 14236.2  on 10698  degrees of freedom
## Residual deviance:  8081.2  on 10689  degrees of freedom
## AIC: 8101.2
##
## Number of Fisher Scoring iterations: 6
```

#Reduced truncated MOV logistic regression model

```
season_glm_MOVTruncRed <- glm(
  home_win ~
    first_downs_diff_season +
    third_down_rate_diff_season +
    mov_trunc_diff_season +
    yards_allowed_diff_season +
    first_downs_diff_season * third_down_rate_diff_season +
    first_downs_diff_season * mov_trunc_diff_season +
    third_down_rate_diff_season * yards_allowed_diff_season
  ,
  data = train_data,
  family = binomial
)
```

```
summary(season_glm_MOVTruncRed)
```

```
##
## Call:
## glm(formula = home_win ~ first_downs_diff_season + third_down_rate_diff_season +
##     mov_trunc_diff_season + yards_allowed_diff_season + first_downs_diff_season *
##     third_down_rate_diff_season + first_downs_diff_season * mov_trunc_diff_season +
##     third_down_rate_diff_season * yards_allowed_diff_season,
##     family = binomial, data = train_data)
##
## Coefficients:
##                                     Estimate Std. Error
## (Intercept)                        0.4532407  0.0312818
## first_downs_diff_season              0.0051995  0.0103692
## third_down_rate_diff_season          -0.0704718  0.4972145
## mov_trunc_diff_season                0.9352664  0.0236569
## yards_allowed_diff_season            0.0002967  0.0004867
## first_downs_diff_season:third_down_rate_diff_season -0.0391414  0.0921702
## first_downs_diff_season:mov_trunc_diff_season        0.0089615  0.0052733
## third_down_rate_diff_season:yards_allowed_diff_season -0.0097316  0.0054304
##                                     z value Pr(>|z|)
## (Intercept)                        14.489    <2e-16 ***
## first_downs_diff_season              0.501    0.6161
## third_down_rate_diff_season          -0.142    0.8873
## mov_trunc_diff_season                39.535    <2e-16 ***
## yards_allowed_diff_season            0.609    0.5422
## first_downs_diff_season:third_down_rate_diff_season  -0.425    0.6711
## first_downs_diff_season:mov_trunc_diff_season        1.699    0.0892 .
## third_down_rate_diff_season:yards_allowed_diff_season -1.792    0.0731 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
```

```
##      Null deviance: 14236.2  on 10698  degrees of freedom
## Residual deviance:  8074.6  on 10691  degrees of freedom
## AIC: 8090.6
##
## Number of Fisher Scoring iterations: 6
```

4.1 Binned Probabilities and Confusion Matrix for Full Non MOV Truncated Model

This section tests the models on the test data set and displays the results. The binned test accuracy graph displays the test accuracy across different predicted win probabilities. With a perfect model, I would expect the test accuracy to match the predicted probability. For example, if the true probability of a win is 50%, a perfect model would correctly predict the outcome 50% of the time. Therefore, the binned graph of an accurate model should have a U shape indicating the highest and lowest win probabilities will be the best predicted.

The confusion matrix of the models displays the number of correct and incorrect classifications by win or loss.

```
#Applies the model to the test set and calculates the test accuracy.
```

```
probs_full <- predict(season_glm_full, test_data, type = "response")
pred_class_full <- ifelse(probs_full > 0.5, 1, 0)

mean(pred_class_full == test_data$home_win)
```

```
## [1] 0.788409
```

```
#Displays the test accuracy by predicted probability ranges
```

```
preds <- data.frame(
  home_win = test_data$home_win,
  pred_prob = probs_full
)
```

```

preds <- preds %>%
  mutate(
    prob_bin = cut(
      pred_prob,
      breaks = seq(0, 1, by = 0.1),
      include.lowest = TRUE,
      right = FALSE,
      labels = c("0-10%", "10-20%", "20-30%", "30-40%", "40-50%",
                  "50-60%", "60-70%", "70-80%", "80-90%", "90-100%")
    ),
    pred_class = ifelse(pred_prob > 0.5, 1, 0),
    correct = ifelse(pred_class == home_win, 1, 0)
  )

bin_accuracy <- preds %>%
  group_by(prob_bin) %>%
  summarise(
    accuracy = mean(correct),
    count = n(),
    .groups = "drop"
  )

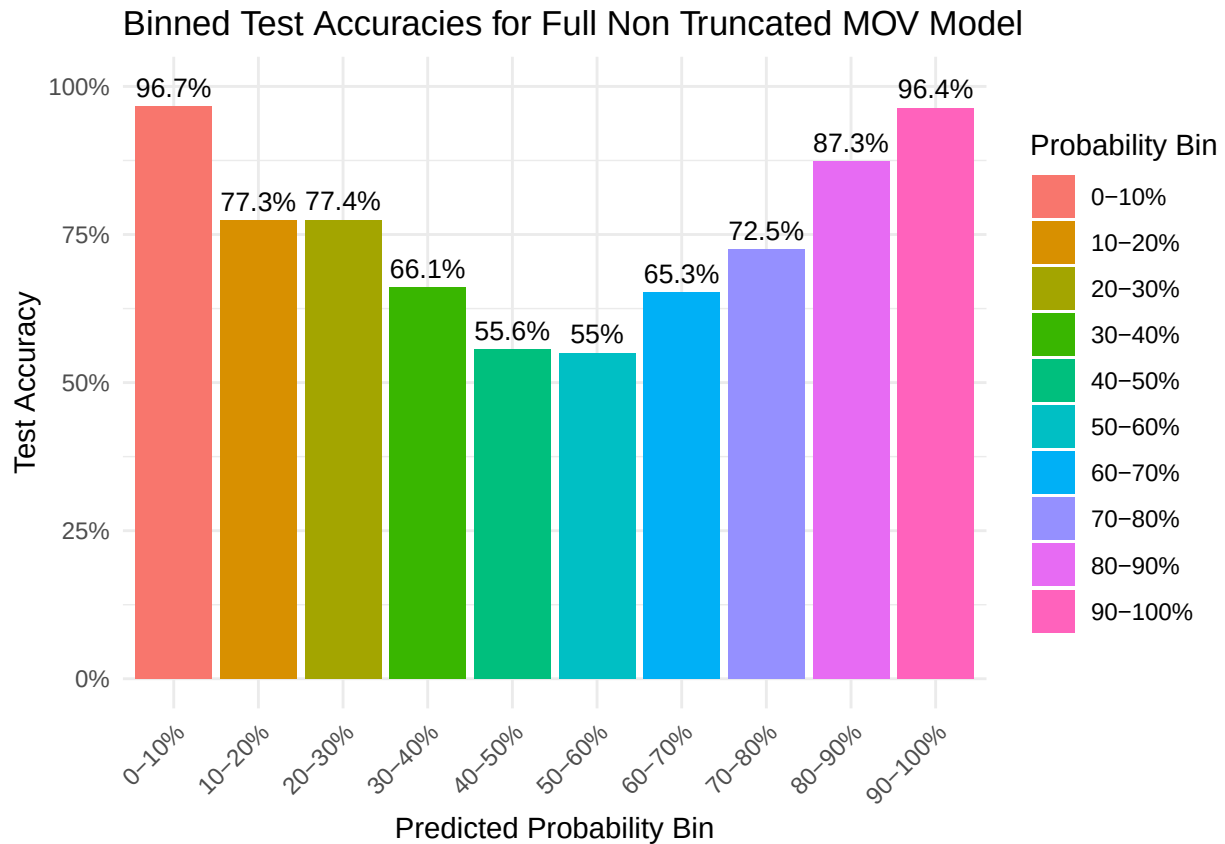
ggplot(bin_accuracy, aes(x = prob_bin, y = accuracy, fill = prob_bin)) +
  geom_col() +
  geom_text(aes(label = paste0(round(accuracy * 100, 1), "%")),
            vjust = -0.5, size = 3.5) +
  scale_y_continuous(
    labels = scales::percent_format(accuracy = 1),
    limits = c(0, 1)
  ) +
  labs(
    x = "Predicted Probability Bin",
    y = "Test Accuracy",
    fill = "Probability Bin",

```

```

title = "Binned Test Accuracies for Full Non Truncated MOV Model"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



#Displays confusion matrix for the model

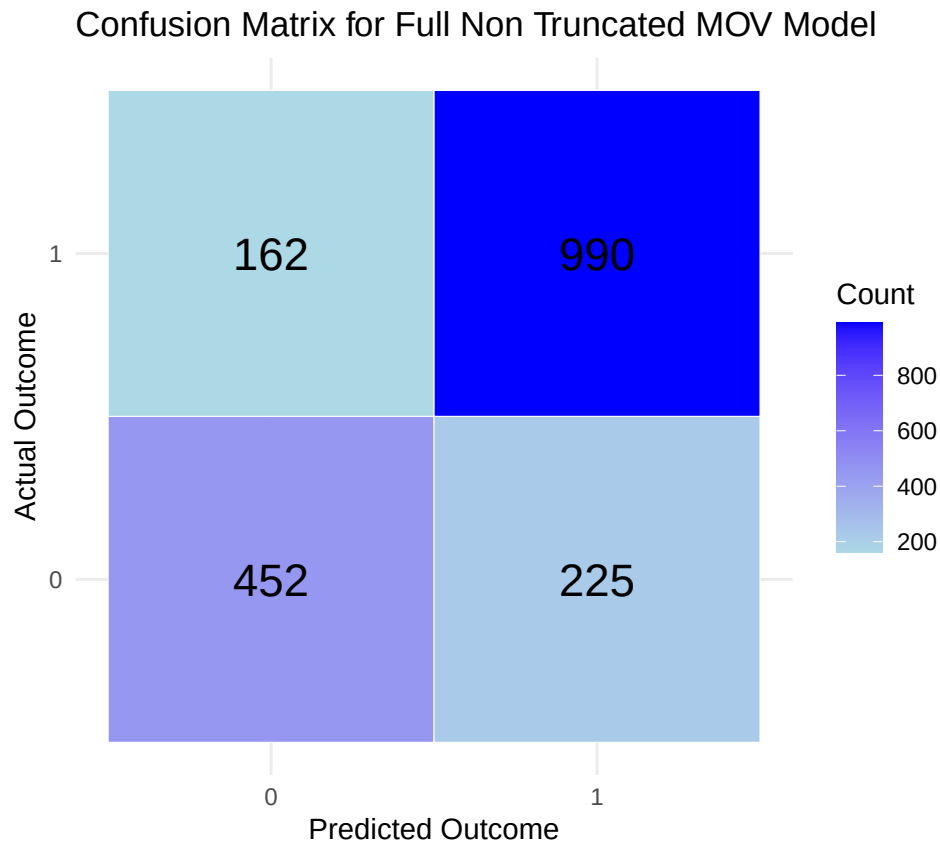
```

conf_matrix <- data.frame(
  Actual = factor(test_data$home_win, levels = c(0, 1)),
  Predicted = factor(ifelse(probs_full >= 0.5, 1, 0), levels = c(0, 1))
) %>%
  count(Actual, Predicted)

ggplot(conf_matrix, aes(x = Predicted, y = Actual, fill = n)) +
  geom_tile(color = "white") +

```

```
geom_text(aes(label = n), size = 6) +
scale_fill_gradient(low = "lightblue", high = "blue") +
labs(x = "Predicted Outcome", y = "Actual Outcome", fill = "Count", title = "Confusion Matrix") +
theme_minimal() +
coord_fixed()
```



4.2 Binned Probabilities and Confusion Matrix for Full MOV Truncated Model

```
probs_trunc <- predict(season_glm_MOVTrunc, test_data, type = "response")
pred_class_trunc <- ifelse(probs_trunc > 0.5, 1, 0)

mean(pred_class_trunc == test_data$home_win)
```

```
## [1] 0.8168398
```



```

preds <- data.frame(
  home_win = test_data$home_win,
  pred_prob = probs_trunc
)

preds <- preds %>%
  mutate(
    prob_bin = cut(
      pred_prob,
      breaks = seq(0, 1, by = 0.1),
      include.lowest = TRUE,
      right = FALSE,
      labels = c("0-10%", "10-20%", "20-30%", "30-40%", "40-50%",
                 "50-60%", "60-70%", "70-80%", "80-90%", "90-100%")
    ),
    pred_class = ifelse(pred_prob > 0.5, 1, 0),
    correct = ifelse(pred_class == home_win, 1, 0)
  )

bin_accuracy <- preds %>%
  group_by(prob_bin) %>%
  summarise(
    accuracy = mean(correct),
    count = n(),
    .groups = "drop"
  )

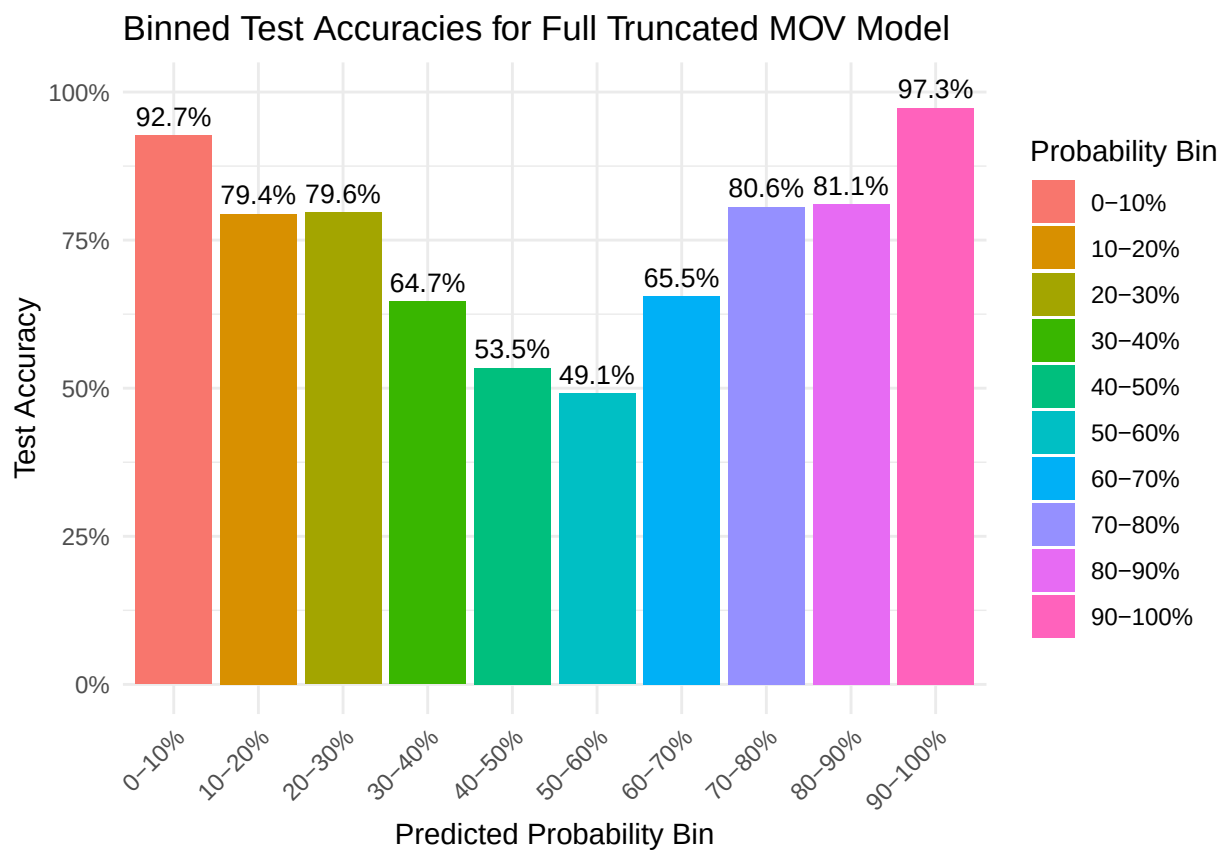
ggplot(bin_accuracy, aes(x = prob_bin, y = accuracy, fill = prob_bin)) +
  geom_col() +
  geom_text(aes(label = paste0(round(accuracy * 100, 1), "%")),
            vjust = -0.5, size = 3.5) +
  scale_y_continuous(
    labels = scales::percent_format(accuracy = 1),
    limits = c(0, 1)
  )

```

```

) +
labs(
  x = "Predicted Probability Bin",
  y = "Test Accuracy",
  fill = "Probability Bin",
  title = "Binned Test Accuracies for Full Truncated MOV Model"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

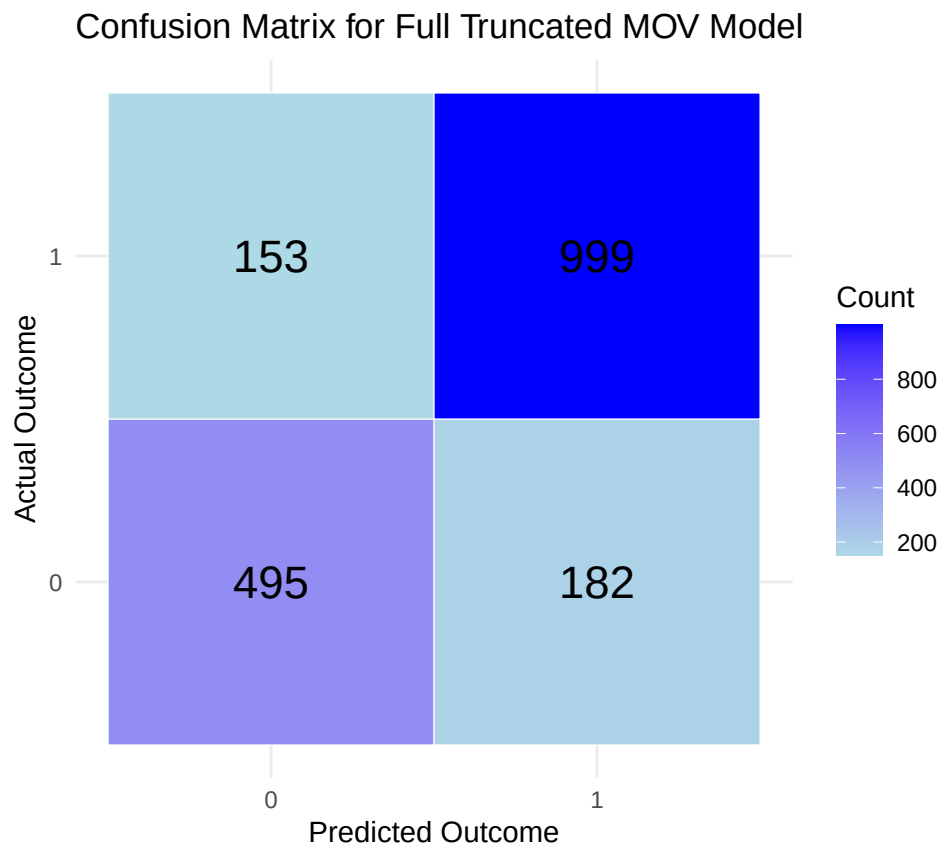


```

conf_matrix <- data.frame(
  Actual = factor(test_data$home_win, levels = c(0, 1)),
  Predicted = factor(ifelse(probs_trunc >= 0.5, 1, 0), levels = c(0, 1))
) %>%
  count(Actual, Predicted)

```

```
ggplot(conf_matrix, aes(x = Predicted, y = Actual, fill = n)) +
  geom_tile(color = "white") +
  geom_text(aes(label = n), size = 6) +
  scale_fill_gradient(low = "lightblue", high = "blue") +
  labs(x = "Predicted Outcome", y = "Actual Outcome", fill = "Count", title = "Confusion Matrix") +
  theme_minimal() +
  coord_fixed()
```



4.3 Binned Probabilities and Confusion Matrix for Reduced Non MOV Truncated Model

```
probs_truncRed <- predict(season_glm_MOVTruncRed, test_data, type = "response")
pred_class_truncRed <- ifelse(probs_truncRed > 0.5, 1, 0)
```

```
mean(pred_class_truncRed == test_data$home_win)
```

```
## [1] 0.8190268
```

```
preds <- data.frame(  
  home_win = test_data$home_win,  
  pred_prob = probs_truncRed  
)
```

```
preds <- preds %>%  
  mutate(  
    prob_bin = cut(  
      pred_prob,  
      breaks = seq(0, 1, by = 0.1),  
      include.lowest = TRUE,  
      right = FALSE,  
      labels = c("0-10%", "10-20%", "20-30%", "30-40%", "40-50%",  
                 "50-60%", "60-70%", "70-80%", "80-90%", "90-100%")  
    ),  
    pred_class = ifelse(pred_prob > 0.5, 1, 0),  
    correct = ifelse(pred_class == home_win, 1, 0)  
  )
```

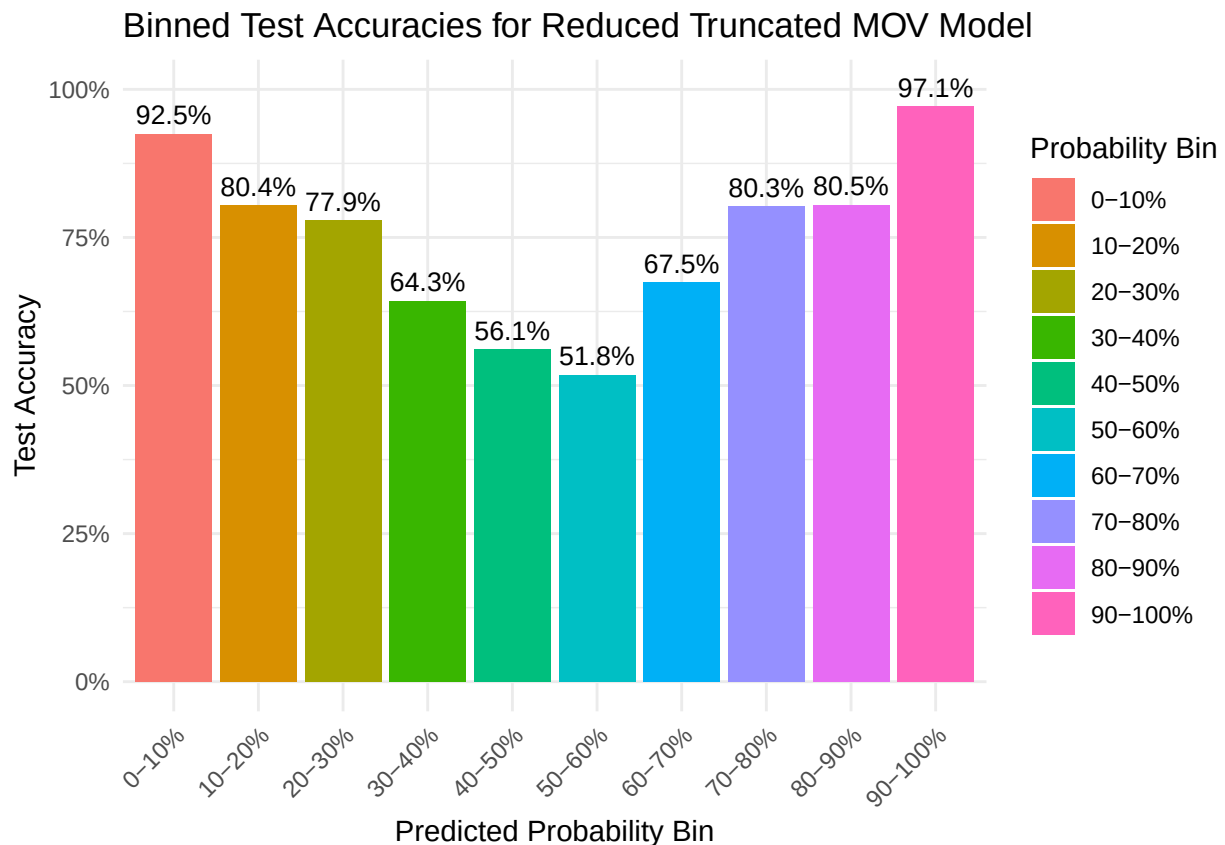
```
bin_accuracy <- preds %>%  
  group_by(prob_bin) %>%  
  summarise(  
    accuracy = mean(correct),  
    count = n(),  
    .groups = "drop"  
  )
```

```
ggplot(bin_accuracy, aes(x = prob_bin, y = accuracy, fill = prob_bin)) +  
  geom_col() +  
  geom_text(aes(label = paste0(round(accuracy * 100, 1), "%"))),
```

```

      vjust = -0.5, size = 3.5) +
scale_y_continuous(
  labels = scales::percent_format(accuracy = 1),
  limits = c(0, 1)
) +
labs(
  x = "Predicted Probability Bin",
  y = "Test Accuracy",
  fill = "Probability Bin",
  title = "Binned Test Accuracies for Reduced Truncated MOV Model"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

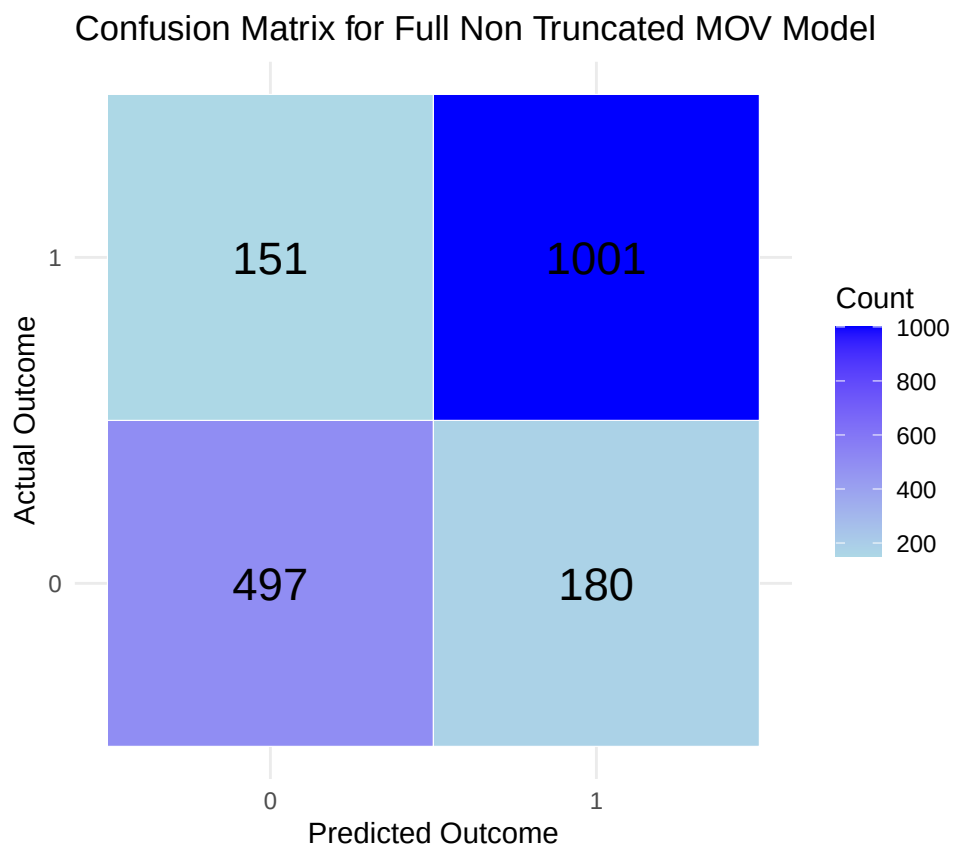


```

conf_matrix <- data.frame(
  Actual = factor(test_data$home_win, levels = c(0, 1)),
  Predicted = factor(ifelse(probs_truncRed >= 0.5, 1, 0), levels = c(0, 1))
) %>%
  count(Actual, Predicted)

ggplot(conf_matrix, aes(x = Predicted, y = Actual, fill = n)) +
  geom_tile(color = "white") +
  geom_text(aes(label = n), size = 6) +
  scale_fill_gradient(low = "lightblue", high = "blue") +
  labs(x = "Predicted Outcome", y = "Actual Outcome", fill = "Count", title = "Confusion Matrix") +
  theme_minimal() +
  coord_fixed()

```



5 Decision Tree Model

In addition to logistic regression, I fit a decision tree model to predict wins using the same set of season-level differential features. A decision tree works by recursively splitting the training data into increasingly homogeneous subgroups based on threshold values of individual predictors. A decision tree can capture non-linear relationships and interactions between features automatically, making it a useful comparison for whether the season differentials interact in ways a linear model would miss. However, decision trees grown to full depth tend to overfit the training data, carving out increasingly specific and idiosyncratic splits that reflect noise in individual games rather than generalizable patterns—this is especially a risk here given a relatively small number of predictors but potentially many possible split points across seasons. To address this, I also fit a pruned decision tree, which reduces the size of the fully-grown tree by removing splits that don't meaningfully improve predictive performance.

```
#Creates the unpruned decision tree model
```

```
library(rpart)
```

```
library(rpart.plot)
```

```
set.seed(123)
```

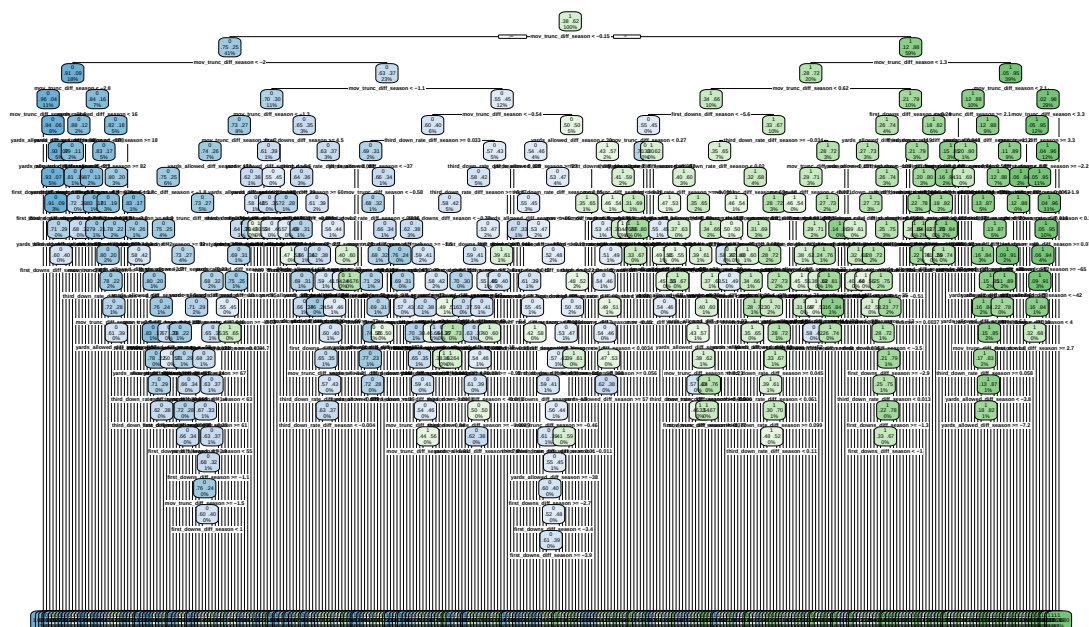
```
tree_model <- rpart(home_win ~  
  first_downs_diff_season +  
    third_down_rate_diff_season +  
  mov_trunc_diff_season +  
  yards_allowed_diff_season,
```

```
  data = train_data, method = "class", control = rpart.control(cp = 0.00001))
```

```
rpart.plot(tree_model, type = 2, extra = 104, fallen.leaves = TRUE, main = "Classification")
```

```
## Warning: labs do not fit even at cex 0.15, there may be some overplotting
```

Classification Tree College Football Wins



#Calculates the test accuracy of the model

```
pred_classTree <- predict(tree_model, newdata = test_data, type = "class")

mean(pred_classTree == test_data$home_win)
```

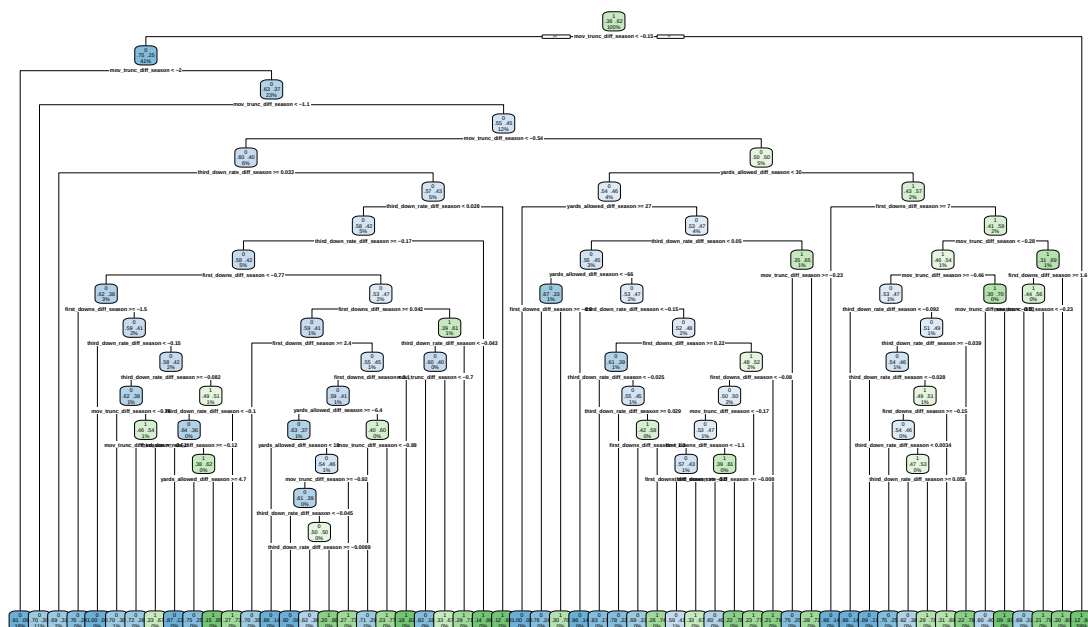
```
## [1] 0.7758338
```

#Creates pruned decision tree with optimized cp value

```
pruned_tree <- prune(tree_model, cp = .0005700790)
rpart.plot(pruned_tree, type = 2, extra = 104, fallen.leaves = TRUE, main = "Pruned Clas
```

```
## Warning: labs do not fit even at cex 0.15, there may be some overplotting
```


Pruned Classification Tree College Football Wins



#Calculates the accuracy of the model

```
pred_classPrune <- predict(pruned_tree, newdata = test_data, type = "class")

mean(pred_classPrune == test_data$home_win)
```

```
## [1] 0.8173866
```

6 K Nearest Neighbors

I also fit a k-nearest neighbors model to predict wins using the same season-level differenced features. Rather than estimating coefficients or building explicit decision rules, KNN classifies each test game by identifying the k most similar games in the training data and predicting the majority class among those neighbors. For example, with $k = 5$, a test game would be classified as a home win if at least 3 of its 5 closest games in the training set were

wins. Because k-nearest neighbors relies on distance calculations, I scaled the predictors before fitting the model so that features on different scales (e.g., yards allowed vs. conversion rate) would contribute proportionally rather than being dominated by variables with naturally larger numeric ranges. I also ran the model at different k values and chose the one that produced the highest test accuracy as the final model.

```
#Creates the k-nearest neighbors model with k of 13.5
library(dplyr)
library(class)
library(tidyr)
set.seed(123)

predictors <- c(
  "first_downs_diff_season",
  "third_down_rate_diff_season",
  "mov_trunc_diff_season",
  "yards_allowed_diff_season"
)

train_clean <- train_data %>%
  select(all_of(predictors), home_win) %>%
  drop_na()

test_clean <- test_data %>%
  select(all_of(predictors), home_win) %>%
  drop_na()

train_x <- train_clean %>% select(all_of(predictors)) %>% as.matrix()
test_x <- test_clean %>% select(all_of(predictors)) %>% as.matrix()

train_y <- as.factor(train_clean$home_win)
test_y <- test_data$home_win
```

```

train_x_scaled <- scale(train_x)

test_x_scaled <- scale(
  test_x,
  center = attr(train_x_scaled, "scaled:center"),
  scale = attr(train_x_scaled, "scaled:scale")
)

knn_pred <- knn(
  train = train_x_scaled,
  test = test_x_scaled,
  cl = train_y,
  k = 13.5
)

accuracy <- mean(knn_pred == test_clean$home_win)
accuracy

```

```
## [1] 0.8151996
```

```
#Displays the test accuracy at different k values
```

```

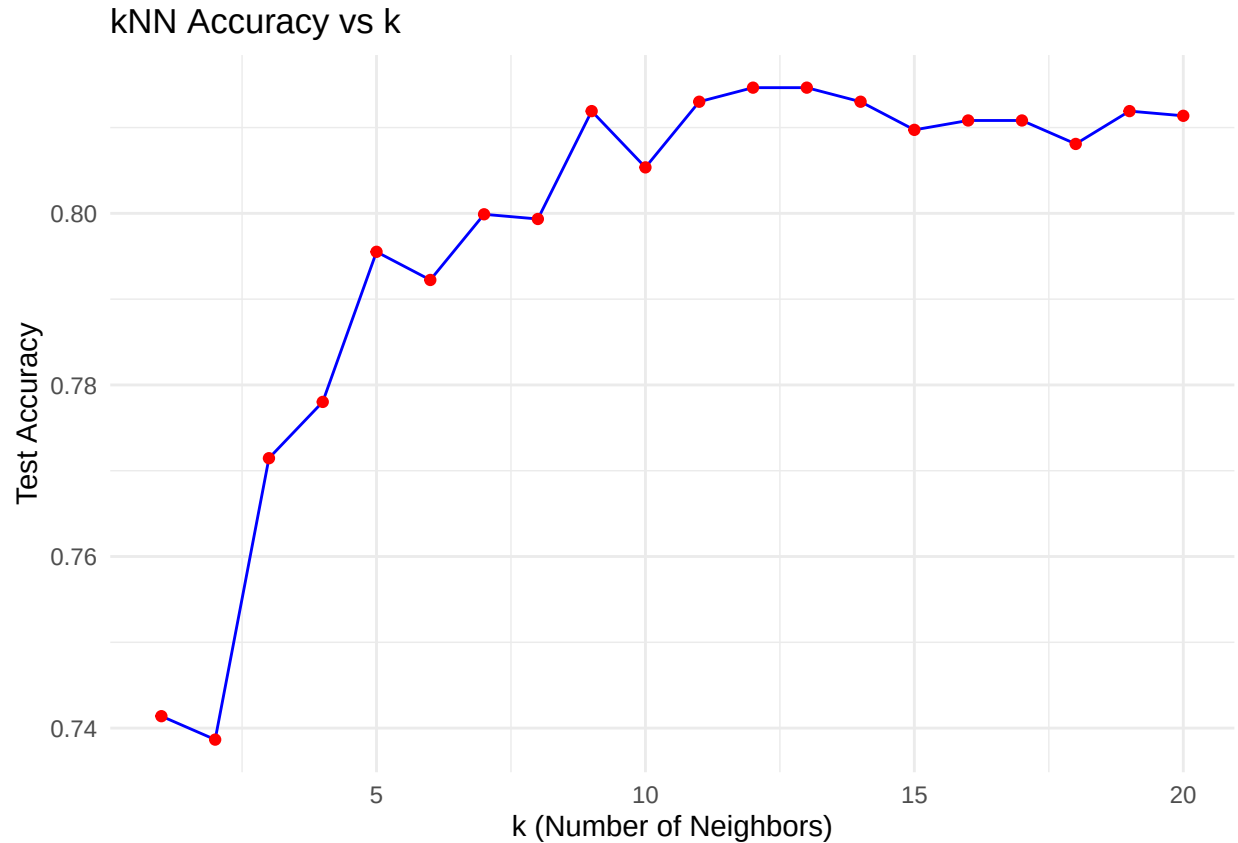
k_vals <- seq(1, 20, 1)
accuracy <- numeric(length(k_vals))

for (i in seq_along(k_vals)) {
  pred <- knn(train = train_x_scaled, test = test_x_scaled, cl = train_y, k = k_vals[i])
  accuracy[i] <- mean(pred == test_y)
}

data.frame(k = k_vals, accuracy = accuracy) %>%
  ggplot(aes(x = k, y = accuracy)) +
  geom_line(color = "blue") +

```

```
geom_point(color = "red") +
labs(x = "k (Number of Neighbors)", y = "Test Accuracy", title = "kNN Accuracy vs k")
theme_minimal()
```



7 Model Accuracy Comparison

```
model_results <- data.frame(
  Model = c("Non Truncated MOV Full Model", "Truncated MOV Full Model", "Truncated MOV R
  Test_Accuracy = c("78.84", "81.68", "81.79", "78.51", "81.19", "81.52")
)

model_results <- model_results %>%
  arrange(desc(Test_Accuracy))

model_results
```

		Model Test_Accuracy
## 1	Truncated MOV Reduced Model	81.79
## 2	Truncated MOV Full Model	81.68
## 3	K-nearest neighbors (k = 13.5)	81.52
## 4	Pruned Decision Tree (cp = 0.00057)	81.19
## 5	Non Truncated MOV Full Model	78.84
## 6	Decision Tree (cp = 0.00001)	78.51

8 Championship Game Predictions with the Reduced Truncated MOV Model

Finally, I will use the reduced MOV truncated logistic regression model to predict the 2025 conference championship games.

```
#JMU Troy Prediction
new_game <- data.frame(
  first_downs_diff_season = 4.9166666,
  third_down_rate_diff_season = 0.0846663931,
  mov_trunc_diff_season = 2,
  yards_allowed_diff_season = -126.4
)

pred_prob <- predict(season_glm_MOVTruncRed, newdata = new_game, type = "response")
pred_prob
```

```
##          1
## 0.9228595
```

```
#North Texas Tulane Prediction
new_game <- data.frame(
  first_downs_diff_season = -3.58333334,
  third_down_rate_diff_season = -0.0453329644,
  mov_trunc_diff_season = -0.583333333,
```

```

yards_allowed_diff_season    = -23.5
)

pred_prob <- predict(season_glm_MOVTruncRed, newdata = new_game, type = "response")
pred_prob

```

```

##          1
## 0.4718466

```

```

#Texas Tech BYU Prediction
new_game <- data.frame(
  first_downs_diff_season    = 1.91666666,
  third_down_rate_diff_season = 0.09950077461,
  mov_trunc_diff_season      = 0.1666666663,
  yards_allowed_diff_season  = -82.8
)

pred_prob <- predict(season_glm_MOVTruncRed, newdata = new_game, type = "response")
pred_prob

```

```

##          1
## 0.6599526

```

```

# Texas A&M Miami Game

new_game <- data.frame(
  first_downs_diff_season    = -0.1666666,
  third_down_rate_diff_season = -0.064685315,
  mov_trunc_diff_season      = 0.0833333,
  yards_allowed_diff_season  = 46.4
)

pred_prob <- predict(season_glm_MOVTruncRed, newdata = new_game, type = "response")
pred_prob

```

```
##          1
## 0.6404455
```

```
# Oklahoma Alabama Game
```

```
new_game <- data.frame(
  first_downs_diff_season    = -3.1923077,
  third_down_rate_diff_season = -0.056446781,
  mov_trunc_diff_season      = 0.35,
  yards_allowed_diff_season  = -19.5
)

pred_prob <- predict(season_glm_MOVTruncRed, newdata = new_game, type = "response")
pred_prob
```

```
##          1
## 0.6757703
```

```
#Virginia Duke Prediction
```

```
new_game <- data.frame(
  first_downs_diff_season    = -1.083333337,
  third_down_rate_diff_season = 0.09633453791,
  mov_trunc_diff_season      = 1.833333333,
  yards_allowed_diff_season  = -109.9
)

pred_prob <- predict(season_glm_MOVTruncRed, newdata = new_game, type = "response")
pred_prob
```

```
##          1
## 0.9013387
```

```

#Decision boundary graph comparing truncated
#MOV difference and first down difference

x_feature <- "mov_trunc_diff_season"
y_feature <- "first_downs_diff_season"

grid <- expand.grid(
  mov_trunc_diff_season = seq(min(train_data$mov_trunc_diff_season),
                              max(train_data$mov_trunc_diff_season), length.out = 200),
  first_downs_diff_season = seq(min(train_data$first_downs_diff_season),
                                 max(train_data$first_downs_diff_season), length.out = 200)
)

grid$third_down_rate_diff_season <- mean(train_data$third_down_rate_diff_season)
grid$yards_allowed_diff_season <- mean(train_data$yards_allowed_diff_season)

grid$prob <- predict(season_glm_MOVTruncRed, newdata = grid, type = "response")
grid$pred <- ifelse(grid$prob > 0.5, 1, 0)
grid$pred <- factor(grid$pred, levels = c(0,1), labels = c("Loss","Win"))
train_data$home_win <- factor(train_data$home_win, levels = c(0,1), labels = c("Loss","Win"))

ggplot() +
  geom_tile(data = grid, aes_string(x = x_feature, y = y_feature, fill = "pred"), alpha = 0.5) +
  geom_point(data = train_data, aes_string(x = x_feature, y = y_feature, color = "home_win")) +
  scale_fill_manual(values = c("Loss" = "salmon", "Win" = "mediumseagreen")) +
  scale_color_manual(values = c("Loss" = "red", "Win" = "springgreen3")) +
  labs(
    fill = "Predicted Class",
    color = "Actual Outcome",
    title = "Truncated MOV Reduced Logistic Regression Decision Boundary",
  )

```



```

x = "Average Truncated Margin of Victory Difference",
y = "Average First Downs Difference"
) +
theme_minimal()

```

```

## Warning: 'aes_string()' was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with 'aes()'.
## i See also 'vignette("ggplot2-in-packages")' for more information.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```

